

Inanovai Assistant

One assistant, reachable from every chat app we use.

Microsoft Teams

LIVE

Telegram

LIVE • @INANOVAI_BOT

WhatsApp

NEEDS A NUMBER

Model

LLAMA 3.3 70B

Hosting

AZURE • FREE TIER

What it does

Ask it a question in a chat and it answers. It keeps the last five exchanges of each conversation, so follow-ups like *"and what about that one?"* land properly.

It cannot reach mail, files or calendars. It reads messages addressed to it and replies. Nothing is written to disk, which is also why nobody — including us — can read anyone's conversation afterwards.

The one idea worth knowing

The project is two layers that know nothing about each other. Every other decision follows from this, and it is why adding Telegram took a day rather than a rewrite.

`app.py` • `bot.py` • `channels/`

Speaks Teams, Telegram and WhatsApp. Holds no intelligence.

`agent.py`

Holds the intelligence. Has never heard of a chat app.

They meet at one method: `ask(message, context) → str` . Text in, text out. Changing the model is one new class. Adding a chat app is one new file in `channels/` . Neither side notices the other changed.

Where a message lands

ROUTE	HANDLED BY	ARRIVES VIA
/api/messages	bot.py	Azure Bot Service
/telegram/webhook	channels/telegram.py	Telegram Bot API
/whatsapp/webhook	channels/whatsapp.py	Meta Cloud API

All three funnel into the same agent. Memory is filed by platform — `telegram:555` , `whatsapp:9199...` — so two conversations can never see each other, even across apps.

Where it runs

PIECE	NAME
Resource group	teams-bot-rg · Central India
Web app	inanovai-teams-bot.azurewebsites.net
Service plan	inanovai-bot-plan · Linux F1
Azure Bot	inanovai-teams-bot · F0

Secrets sit in App Service application settings, never in the deployed package. Azure and the bot registration are both free tiers, so the model API is the only thing that costs money.

Four commands you will actually use

Is it up, and which channels are on?

```
curl https://inanovai-teams-bot.azurewebsites.net/
```

Run every test. No keys, no network, no cost.

```
.venv\Scripts\python.exe run_tests.py
```

Ship a change.

```
az webapp deploy --resource-group teams-bot-rg \  
  --name inanovai-teams-bot --src-path deploy.zip --type zip
```

Watch it think.

```
az webapp log tail --resource-group teams-bot-rg --name inanovai-teams-bot
```

Traps we already walked into

01 A stale environment variable quietly beat the real key

A leftover `OPENAI_API_KEY=your-api-key-here` in the Windows user environment overrode `.env`, and the 401s looked exactly like a bad key. `config.py` now loads with `override=True` so the project's own file wins. Leave that flag alone.

02 An app registration is not an identity

Creating the Entra ID app does not create the service principal it signs in with. Messages arrived, replies never went out, and the log said `AADSTS7000229`. One command fixed it: `az ad sp create --id <app-id>`. Any new registration needs that same second step.

03 One WhatsApp number points at one webhook

Our number already feeds the ERP integration. Attaching the bot to it would redirect the ERP's messages. The bot needs a second number on the same Business Account — same billing, same verification, separate destination.

04 The free tier sleeps

After twenty idle minutes the first message waits about twenty seconds while the app wakes up. Everything after that is quick. Switching the plan to B1 removes it, with no redeploy.

What is next

Prashanth to approve the pending Teams app so it shows up under *Built for your org*. A second phone number to finish WhatsApp. After that, Slack and a web widget are each a single file in `channels/` — the layer split has already paid for them.